

Análise de Código Java

LinkedListCycle — Detecção de Ciclo em Lista Ligada

Código Analisado

```
public class LinkedListCycle {  
    static class ListNode { int val; ListNode next; ListNode(int v){val=v;} }  
  
    public static boolean hasCycle(ListNode head) {  
        ListNode slow = head, fast = head;  
        while (fast != null && fast.next != null) {  
            slow = slow.next;  
            fast = fast.next.next;  
            if (slow == fast) return true;  
        }  
        return false;  
    }  
  
    public static void main(String[] args) {  
        ListNode n1=new ListNode(3), n2=new ListNode(2),  
            n3=new ListNode(0), n4=new ListNode(-4);  
        n1.next=n2; n2.next=n3; n3.next=n4; n4.next=n2; // ciclo  
        System.out.println(hasCycle(n1)); // true  
        ListNode a=new ListNode(1); a.next=new ListNode(2);  
        System.out.println(hasCycle(a)); // false  
    }  
}
```

■ Objetivo Geral

O código verifica se uma lista ligada simples contém um ciclo, ou seja, se algum nó aponta de volta para um nó anterior, criando um loop infinito. O método **hasCycle** retorna **true** se houver ciclo e **false** caso contrário.

■■ Algoritmo: Floyd's Cycle Detection (Tartaruga e Lebre)

Dois ponteiros percorrem a lista simultaneamente: **slow** avança um nó por iteração e **fast** avança dois. Se existir ciclo, o ponteiro rápido eventualmente alcança o lento dentro do ciclo e eles se encontram (mesmo endereço de memória). Se a lista é acíclica, **fast** ou **fast.next** atingem **null** e o loop termina.

■ Conceitos Java Utilizados

A solução emprega **classe interna estática** (*static nested class*) para modelar o nó da lista, **referências de objeto** como ponteiros e comparação por identidade com `==` (endereço em memória, não igualdade de valor). Não são usadas coleções, generics ou threads — a implementação é deliberadamente minimalista.

■ Complexidade

Complexidade de tempo: **$O(n)$** , pois no pior caso os ponteiros percorrem a lista inteira. Complexidade de espaço: **$O(1)$** , já que apenas dois ponteiros auxiliares são utilizados, independentemente do tamanho da lista — vantagem clara sobre abordagens com HashSet ($O(n)$ extra).

■■ Decisão de Design e Limitações

Usar `==` compara referências (identidade), o que é correto aqui pois interessa saber se os ponteiros apontam para o *mesmo objeto*. Uma limitação é que o código não detecta *onde* o ciclo começa — apenas confirma sua existência. Para localizar o nó de entrada do ciclo, seria necessária uma segunda fase do algoritmo de Floyd.

Aspecto	Detalhe
Problema	Detecção de ciclo em lista ligada
Algoritmo	Floyd's Cycle Detection (Two-Pointer)
Complexidade Tempo	$O(n)$
Complexidade Espaço	$O(1)$
Conceitos Java	Classe interna estática, referências, comparação <code>==</code>
Saída (exemplo)	true (com ciclo) / false (sem ciclo)